

TESTKONZEPT für ATLAS

Aufbau von Testinfrastruktur und automatisierten Tests

Angular 19 + ASP.NET Core WebApp

Projektphase

Aufbau Testinfrastruktur

Zuständigkeiten

Student

Datum

19.06.2026

Verfasser

Kristanna Pfeil

1 Zweck und Zielstellung

Die WebApp-Lösung ATLAS (Frontend: Angular 19 mit Angular Material; Backend: ASP.NET Core) verfügt bislang über keine bzw. rudimentäre Unit- und Integrationstests. Dieses Testkonzept definiert eine erste Grundlage für automatisiertes Testen und beschreibt für zentrale Komponenten der Software erste Tests.

1.1 Qualitätsziele

- Etablierung einer stabilen Test-Infrastruktur
- Messbare erhöhte Testabdeckung
- Frühe Fehlererkennung durch automatisierte Tests
- Saubere Architektur um mögliche Erweiterungen effizient integrieren zu können
- (CI/CD-Integration)

2 IST-Zustand

Code Coverage des IST-Zustands durch aktuelle Unit- und Integrationstests im Backend:



Abbildung 1 Backend - Code Coverage Result

Von den fünf Haupt-Services sind nur Experiment-, File- und Search-Service abgedeckt und dabei noch weit unter dem Ziel von einem durchschnittlichen 80% Code-Coverage.

Das Frontend hat noch keine vorhandenen Tests oder Code Coverage.

Für alle weiteren Services und Komponenten aus Backend und Frontend gibt es noch keinerlei Tests oder Code-Coverage.

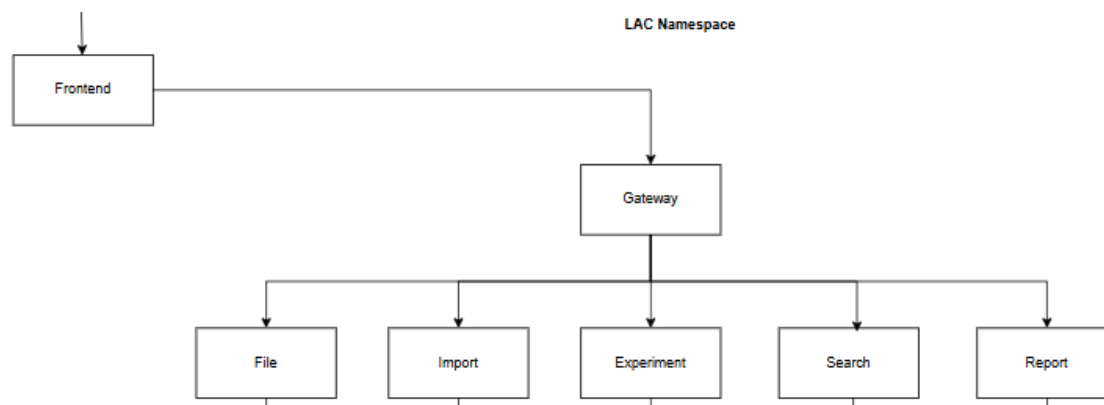


Abbildung 2 Frontend - Backend | Quelle: Confluence

3 Umfang des Testens

3.1 Backend – ASP.NET Core REST-API

Folgende Testkomponenten und Unit-, und Integrationstest dienen als erste Grundlage für zu testende Bereiche. **Diese können sich im Laufe der Entwicklung noch anpassen.**

Im reinen Backend wird hauptsächlich mit Unit- und Integrationstests gearbeitet werden. Weitere Testarten können je nach Bedarf oder Wunsch ergänzt werden.

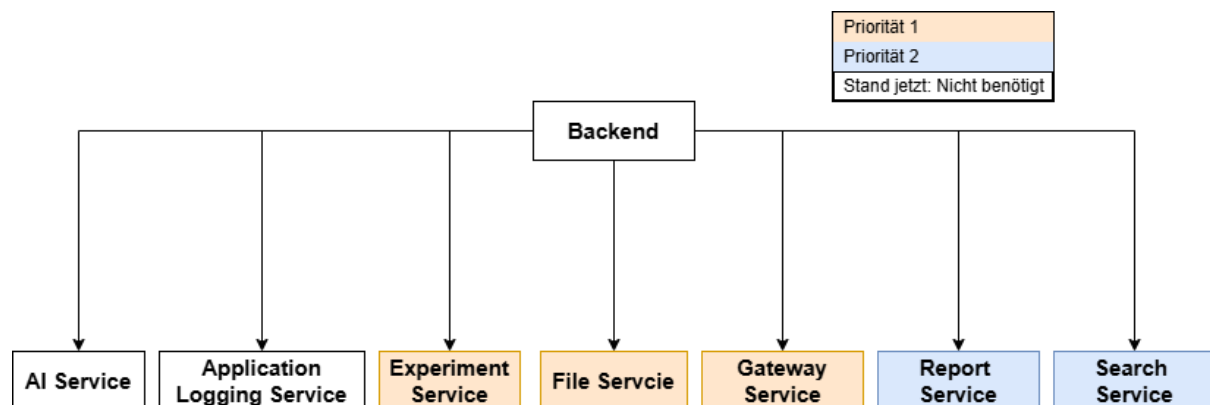


Abbildung 3 Backend Services - Priorität

GatewayService, ExperimentService und FileService sollten zuerst getestet werden, weil sie die kritischsten Prozesse der Anwendung abdecken und mit am meisten Daten arbeiten.

Der GatewayService ist der zentrale Einstiegspunkt für das Frontend und prüft Berechtigungen, der ExperimentService verwaltet die wichtigsten fachlichen Daten wie Experimente, Tasks und Parameter, und der FileService ist für Upload, Speicherung und Abruf von Dateien, Bildern und Reports verantwortlich.

Fehler in diesen Services hätten daher besonders große Auswirkungen auf Sicherheit, Datenqualität und Nutzbarkeit der Anwendung.

Aus diesem Grund werden die Services aus Priorität 1 als erstes abgedeckt.

Priorität 1

Komponente	Zweck	1. Unittest	2. Integrationstest
Experiment Service	Daten-CRU(D): lädt, filtert, erstellt und aktualisiert Experimente und wandelt dabei Datenbank-Entities in Models um.	Experimente, Tasks, Setups, ParameterSets, Result, Validierung, Mapping, Create/Update-Logik, Duplikatsprüfungen Alle Controller und Services abdecken	API + Datenbank: Experimente anlegen, Tasks hinzufügen, Parameter speichern, Ergebnisse abrufen HTTP-Requests an Controller
Gateway Service	Nimmt Frontend-Anfragen entgegen, prüft Berechtigungen	Controller, Autorisierung, Weiterleitung an Services,	End-to-End-API-Calls vom Gateway zu Mock-/Test-Services,

	und leitet sie an die passenden Backend-Services weiter.	Fehlerbehandlung (Logging), DTO-Mapping	Authentifizierung, Rollen-/Permission-Flows
File Service	speichert, sucht, lädt und löscht Dateien über Azure Blob Storage.	Tag-Erzeugung, Content-Type-Erkennung, Dateinamen-/ZIP-Logik, Fehlerfälle	Azure Blob Storage bzw. Storage-Emulator: Upload, Download, Suche per Tags, Löschen, Report-Artefakte

Priorität 2

Komponente	Zweck	1. Unittest	2. Integrationstest
Report Service	Erzeugt Reports mit Excel-, PowerPoint- und Bilddaten und legt sie als ZIP-Datei ab.	Datenaggregation, Rundung, Asset-Auswahl, Bildkompression, Slide-Preview-Logik	Report-Generierung mit Testdatenbank + FileService-Mock: Excel, PowerPoint, Bilder ZIP und Preview-Erzeugung, Upload
Search Service	Durchsucht Parameter- und Experimentdaten anhand von Filtern und liefert tabellarische Suchergebnisse zurück.	Filterlogik, Materialfilter, Messwertfilter, Spaltendefinitionen, Bookmarks/Filter-Buffer	Suche gegen Testdatenbank mit realistischen Experiment-/Parameterdaten

Ergänzende Backend-Komponenten

Komponente	Zweck	Möglicher Testfokus
Database	Enthält Datenbankkontext, Entities, Repositories und Datenzugriff über Entity Framework.	Entity-Mappings, Repository-Methoden, EF-Core-Queries, Beziehungen,
Core	Enthält gemeinsame Models, Enums und Basisstrukturen, die von mehreren Services verwendet werden.	Models, Enums, Mapping-/Extension-Methoden, Berechnungs-/Formatierungslogik, Validierungsregeln
Authorization	Prüft Benutzerrollen, Berechtigungen und den aktuellen Benutzerkontext.	Rollen, Berechtigungen, <u>PermissionsProvider</u> , <u>UserContext</u> , Zugriff auf Experiment-/Kundendaten
Utility	Enthält Hilfsfunktionen wie Umrechnungen und wiederverwendbare technische Logik.	Umrechnungen, Formeln, Rundungen, technische Hilfsfunktionen wie <u>SpeedUnitConversion</u>

Application Logging Service	Sammelt und liefert Anwendungs- und Konsolenlogs der Services.	Logeinträge, Reihenfolge, Max-Entries
AI Service	Script-Tagging, Dateiname, Fehler bei mehrfach gefundenen Scripts	Blob-Storage-Test: Python-Script hochladen, per Name finden und temporär herunterladen

3.2 Frontend – Angular 19 + Angular Material

Die Priorisierung anhand kritischer Prozesse festgelegt. Die Komponenten in **Priorität eins** sind für den Zugriffsschutz, States und den Datenfluss zuständig und damit sind Fehler innerhalb dieser Bereiche kritischer als in **Priorität 2** und **Ergänzende Komponenten**. Diese sind für die Bedienbarkeit, Wiederverwendbarkeit und Hilfsfunktionen zuständig.

Priorität 1

Komponente	Zweck	1. Unittest	2. Integrationstest
features	Enthält die fachlichen Hauptfunktionen wie Experiment Dashboard, Wizard, Suche, Parameterpflege, Setup und Result.	Einzelne Feature-Komponenten wie CreateExperimentComponent, SetupPageComponent, ParametersPageComponent, ResultComponent auf Validierung, State-Änderungen und Methodenlogik testen.	Komplette Flows testen: Experiment laden/anlegen, Task speichern, Setup berechnen, Parameter bearbeiten, Result erfassen und Suche ausführen.
services	Kapselt API-Zugriffe, State-Management und zentrale Geschäftslogik.	Services mit gemockten Abhängigkeiten testen, z.B. State-Reset, Mapper, Fehlerbehandlung und API-Payloads.	Komponenten mit echten Services oder HTTP-Mocks testen, z. B. Laden von Experimenten, Speichern von Parametern oder Suchanfragen.
auth	Verantwortlich für Login, Rollen, Berechtigungen und Authentifizierungs-Header.	AuthGuard, RoleGuard, AdminGuard und authInterceptor mit verschiedenen Rollen, Tokens und Login-Zuständen testen.	Geschützte Routen testen: Benutzer ohne Rolle wird zu /request-access geleitet, Benutzer mit Rolle darf /experiments, /wizard oder /tools öffnen.
guards	Verhindert unerlaubte Navigation, z. B. bei ungespeicherten Änderungen.	UnsavedChangesGuard und ParametersCanDeactivateGuard testen: Navigation erlaubt, blockiert oder Dialog wird geöffnet.	Wizard-Navigation testen: Benutzer mit geänderten Formulardaten bekommt Confirm-Dialog, bevor er den Step verlässt.

Priorität 2

Komponente	Zwecke	1. Unittest	2. Integrationstest
components	Enthält wiederverwendbare UI-Komponenten wie Header, Sidebar, Form-Komponenten, Dialoge, Logs und Request-Access.	AtlasFormInputComponent, AtlasFormSelectComponent, AtlasFormMultiSelectComponent, ImageSelectionPreviewComponent auf Events, Disabled-State, Clear-Funktion und Value-Handling testen.	Komponenten in echten Formularen testen, z. B. Wizard-Formulare mit Atlas-Inputs oder Upload-Vorschau im Result-Bereich.
layout	Stellt Grundlayout, Tools-Layout und Master-Data-Statusanzeige bereit.	MainLayoutComponent auf Sidebar-Toggle und Master-Data-Dialog-Logik testen.	Routing mit Layout testen: geschützte Seiten werden im korrekten Layout geladen, Header/Sidebar bleiben sichtbar.

Ergänzende Frontend-Komponenten

Komponente	Zwecke	1. Unittest	2. Integrationstest
Interfaces	Enthält Datenmodelle, Enums und Type-Definitionen für Frontend-Datenstrukturen.	Enum-Mappings testen.	Prüfen, ob Komponenten und Services erwartete Datenstrukturen korrekt verwenden, z. B. Result-, Setup- oder User-Modelle.
util	Enthält Hilfskomponenten, Dialoge, Pipes und Directives für wiederkehrende UI-Aufgaben.	TruncatePipe, Confirm-Dialoge, Input-Dialoge und kleine Directives isoliert testen.	Dialog-Nutzung in Komponenten testen, z. B. Löschen eines Parameters oder Warnung bei ungespeicherten Änderungen.

3.3 End-to-End Szenarien – Zusammenführen von Frontend und Backend**Backend und Frontend Komponenten.**

3.3.1 Backend – End-to-End Tests

1. GatewayService – Login und Zugriff:

- Benutzer wird authentifiziert und ruft das Dashboard über den GatewayService auf.
- Benutzer mit gültiger Rolle erhält Zugriff auf geschützte Endpunkte.
- Benutzer ohne Berechtigung wird abgewiesen oder auf /request-access weitergeleitet.
- Berechtigungen werden korrekt auf Experimentdaten angewendet.

2. GatewayService – Experimentdaten geschützt anzeigen:

- Benutzer öffnet ein Experiment über den GatewayService.
- Der GatewayService lädt die Experimentdaten vom ExperimentService.
- Nicht erlaubte Felder werden für den Benutzer ausgeblendet oder ersetzt.
- Benutzer sieht nur die Daten, für die er berechtigt ist.

3. ExperimentService – Experiment anlegen.

- Benutzer startet ein neues Experiment.
- Benutzer füllt Pflichtfelder wie Titel, Kunde und Verantwortliche aus.
- Benutzer speichert das Experiment erfolgreich.
- Das Experiment ist anschließend über die Experimentliste abrufbar.

4. ExperimentService – Experiment Task anlegen.

- Benutzer öffnet ein bestehendes Experiment.
- Benutzer legt einen neuen Experiment Task an.
- Benutzer erfasst relevante Task- und Materialdaten.
- Benutzer speichert den Task erfolgreich und sieht ihn im Experiment.

5. ExperimentService – Setup anlegen und berechnen.

- Benutzer legt zu einem Experiment Task ein Setup an.
- Benutzer wählt Maschine, Laser, Optik und relevante Parameter.
- Benutzer startet die Berechnung.
- Benutzer speichert das Setup erfolgreich.

6. FileService – Datei hochladen.

- Benutzer wählt eine Datei oder ein Bild im Experiment aus.
- Die Datei wird über den GatewayService an den FileService übertragen.
- Der FileService speichert die Datei mit passenden Tags.
- Die Datei ist anschließend im Experiment sichtbar und abrufbar

7. FileService – Datei suchen und herunterladen.

- Benutzer öffnet die Dateiansicht eines Experiments.
- Dateien werden anhand der Experiment-ID gefunden.
- Benutzer wählt eine Datei zum Download aus.
- Die Datei wird korrekt heruntergeladen.

8. ReportService – Report erzeugen.

- Benutzer wählt relevante Parameter und Bilder für einen Report aus.
- Benutzer startet die Report-Erstellung.
- Der ReportService erzeugt PowerPoint-, Excel- und Bilddaten.
- Der fertige Report wird als ZIP-Datei gespeichert und ist abrufbar.

9. ReportService – Report-Vorschau anzeigen:

- Benutzer öffnet ein Experiment mit vorhandenem Report.
- Die Slide-Preview wird geladen oder neu erzeugt.
- Benutzer sieht die Vorschau der Report-Folien.
- Fehlerhafte oder fehlende Reportdaten werden verständlich behandelt.

10. SearchService – Suche ausführen:

- Benutzer öffnet die Suche.
- Benutzer setzt Filter für Material, Parameter oder Messwerte.
- Der SearchService liefert passende Suchergebnisse zurück.
- Benutzer sieht die Ergebnisse korrekt in der Tabelle.

11. SearchService – Suchfilter speichern:

- Benutzer konfiguriert Suchfilter und Tabellenspalten.
- Benutzer speichert die Sucheinstellungen.
- Benutzer lädt die Suche später erneut.
- Die gespeicherten Filter und Spalten werden korrekt wiederhergestellt.

3.3.2 Frontend – End-to-End Tests

1. Login und Zugriff

- Benutzer wird authentifiziert und kann das Dashboard öffnen.
- Benutzer ohne Berechtigung wird auf **/request-access** weitergeleitet.

2. Experiment anlegen

- Benutzer startet ein neues Experiment.
- Benutzer füllt Pflichtfelder aus.
- Benutzer speichert das Experiment erfolgreich.
- Validierungsfehler erscheinen bei fehlenden Pflichtfeldern.

3. Experiment Task anlegen

- Benutzer legt einen Experiment Task an.
- Benutzer erfasst relevante Task- und Materialdaten.
- Benutzer speichert den Task erfolgreich.

4. Setup anlegen und berechnen

- Benutzer legt ein Setup an.
- Benutzer wählt Maschine, Laser und Optik.
- Benutzer startet die Berechnung.
- Benutzer speichert das Setup.

5. Parameter bearbeiten

- Parameter-Grid wird geladen.
- Benutzer fügt eine Parameterzeile hinzu.
- Benutzer bearbeitet und speichert die Parameterzeile.

6. Result erfassen

- Benutzer öffnet den Result-Bereich.
- Benutzer trägt Ergebniswerte ein.
- Benutzer lädt ein Bild oder eine Datei hoch.
- Benutzer speichert das Result.

7. Ungespeicherte Änderungen

- Benutzer ändert Daten.
- Benutzer versucht die Seite zu verlassen.
- Warnung erscheint.
- Benutzer bleibt auf der Seite oder bestätigt das Verlassen.

8. Suche ausführen

- Benutzer öffnet die Suche.
- Benutzer setzt zentrale Suchfilter.
- Benutzer führt die Suche aus.
- Benutzer öffnet ein Suchergebnis.

3.4 CI/CD-Pipeline

Für die Anwendung soll es eine Pipeline geben. Im Backend soll es z.B. **pro Service eine eigene Pipeline** geben. Diese Pipelines sollen bei Pull Requests automatisch ausgeführt werden, sodass Änderungen frühzeitig geprüft werden können, bevor sie gemerged werden. Aktuell sind die Pipelines jedoch deaktiviert. Wichtig ist außerdem, dass es sich dabei nicht um reine Test-Pipelines handelt: Neben Tests können dort auch weitere Schritte wie Build, Analyse, Packaging oder Deployment-Vorbereitung enthalten sein. Am Ende der Pipeline können Agents eingesetzt werden, die die jeweiligen Pipeline-Schritte ausführen und die Ergebnisse bereitstellen.

Stand jetzt: Sven möchte die Pipeline bearbeiten bzw. diesen Punkt übernehmen.

4 Teststrategie und Pyramide

4.1 Testpyramide

Ebene	Fokus	Volumen & Geschwindigkeit
Unit Tests	Services, Business-Logik, kleine funktionale Units	70–80% der Tests, sehr schnell (<100ms/Test)
Integrations Tests	API-Endpunkte, DB-Interaktion, Component Zusammenspiel	15–20% der Tests, moderat schnell (<500ms/Test)
E2E / UI Tests	Zentrale User Journeys, kritische Workflows	5–10% der Tests, moderately langsam
Weitere Testarten	Frontend: Cross-Browser-Tests	

4.2 Test-Zeitplan

Meilensteine	Aufgaben	Deliverables
Einarbeitung	Einarbeitung, Architektur-Analyse, Ist-Zustand	Analyse-Dokument, Test-Strategie
Backend	Unit-Tests für jeweiligen Services und Komponenten; Integrationstest mit den Services	Test-Suite,
Frontend	Unit-Tests, Integrationstests, Auth-Tests	Test-Suite, Harnesses, Test-Utilities

Gesamt-System	E2E-Tests, CI-Integration, Optimierung	CI-Pipeline, Reports, Guidelines
Abgabe	Dokumentation, Übergabe, finaler Code-Review	Confluence-Dokumentation

5 Testumgebung und Ressourcen

Das Framework kann im Laufe der Entwicklung angepasst werden.

Backend-Test-Stack

- **xUnit**
- WebApplicationFactory für ASP.NET Core Tests
- **Moq für Mocking von Dependencies**
- **Entity Framework Core (ExperimentService)**
- (Postman: Um APIs und backend Services zu testen)

5.1 Frontend-Test-Stack

- Jest + Angular TestBed
- Angular TestBed für Komponenten/Services
- HttpClientTestingModule für API-Mocking
- Moq für Mocking von Dependencies
- Postman: Um APIs und backend Services zu testen

5.2 End-to-End Tests

- Protractor: Ideal für Angular applications. Es interagiert mit der App wie als wäre es ein User
- Playwright: Login, Wizard, Suche, Grid-Interaktion

5.3 CI/CD-Integration

- (Jenkins)
- (CircleCI)
- Azure Pipelines

6 Ziel

- Testausführungszeit: Unit-Tests <20s, Integration <40s
- Fehlertrends: Fehlerdichte sinkt mit zunehmender Testabdeckung
- CI/CD-Pipeline funktionsfähig und Tests automatisiert

- Mindestens 80% Testabdeckung in Kernbereichen
- Dokumentation für zukünftige Tests und allgemeinen Aufbau verfügbar
- Code-Review und Team-Onboarding abgeschlossen

7 Rollen und Verantwortlichkeiten

Rolle	Verantwortlichkeiten
Student	Umsetzung der gesamten Testinfrastruktur (Unit-, Integration-, E2E-Tests), Dokumentation
Haupt-Entwickler / Dev-Team	Code-Review

8 Qualitätssicherung und kontinuierliche Verbesserung

Während der Implementierung der Tests finden folgende Qualitätssicherungsprozesse statt.

- Kleine, fokussierte Funktionen statt große unübersichtliche Code-Blöcke
- Clean Code:
 - Code Wiederverwendung
 - Sinnvolle Tests – „weniger ist mehr“
 - Keine zusätzlichen Tests die möglichen Code in der Zukunft testen könnten
- Fehler aus Tests dokumentieren
- Nach Meilenstein mit Dev-Team abstimmen und reviewen lassen

9 Dokumenthistorie

Version	Datum	Autor	Änderungsbeschreibung
1.0	Juni 2026	Kristanna Pfeil	Initialer Entwurf basierend auf Projektdokumentation
2.0	29.06.2026	Kristanna Pfeil	Feedback von Karin und Sven integriert